~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
class LFUCache {
    int cap,min_freq; // 核心改进：雷达，时刻追踪最小频率
    // 表1: 核心数据 (Key -> {Value, Freq})
    unordered_map<int, pair<int, int>> key_val;
    // 表2: 频率桶 (Freq -> Keys链表)
    unordered_map<int, list<int>> freq_list;
    // 表3: 快速定位 (Key -> 链表迭代器)，用于O(1)删除
    unordered_map<int, list<int>::iterator> key_iter;
public:
    LFUCache(int capacity) : cap(capacity), min_freq(0) {}
    int get(int key) {
        if (key_val.find(key) == key_val.end()) return -1;
        updateFreq(key);
        return key_val[key].first;
    }
    void put(int key, int value) {
        if (key_val.count(key)) {// 1. Key 已存在：改值 + 升频
            key_val[key].first = value;
            updateFreq(key);
            return;
        }
        if (key_val.size() == cap) {
            // 2.1 满了？去 min_freq 桶里杀掉最老的（尾部）
            int del_key = freq_list[min_freq].back();
            freq_list[min_freq].pop_back();
            // 清理另外两张表
            key_val.erase(del_key);
            key_iter.erase(del_key);
        }
        key_val[key] = {value, 1};// 2.2 插入新皇：频率必为 1，min_freq 重置为 1
        freq_list[1].push_front(key);
        key_iter[key] = freq_list[1].begin();
        min_freq = 1; // 新来的肯定是最小频率
    }
private:
    void updateFreq(int key) {
        int freq = key_val[key].second;// 1. 获取旧频率
        // 2. 从旧频率链表中删除 (利用 iter 表实现 O(1))
        freq_list[freq].erase(key_iter[key]);
        // 3.如果 min_freq 的桶空了，且 min_freq 就是当前频率说明最小频率也要跟着涨了
        if (freq_list[freq].empty() && min_freq == freq) {
            min_freq++;
        }
        freq++;
        key_val[key].second = freq;
        freq_list[freq].push_front(key);// 5. 放入新频率链表的头部
        key_iter[key] = freq_list[freq].begin(); // 更新迭代器
    }
};
```